Manipal University Jaipur

School of Computer Science and Engineering
Department of AI & ML

AIM4142 — MLOps

Research project report

Heliograph: detecting covariate shift and concept drift before retraining

Submitted by

Nancy Verma

Registration number: 23FE10CAI00241

Branch: AIML

Deployed project: https://mlops-project-kohl.vercel.app

GitHub: https://github.com/Nancy20052020/Mlops_project

Screenshots of the site

These pictures are from the deployed project. The home page, the observatory, the field desk, and the protocol page are below.

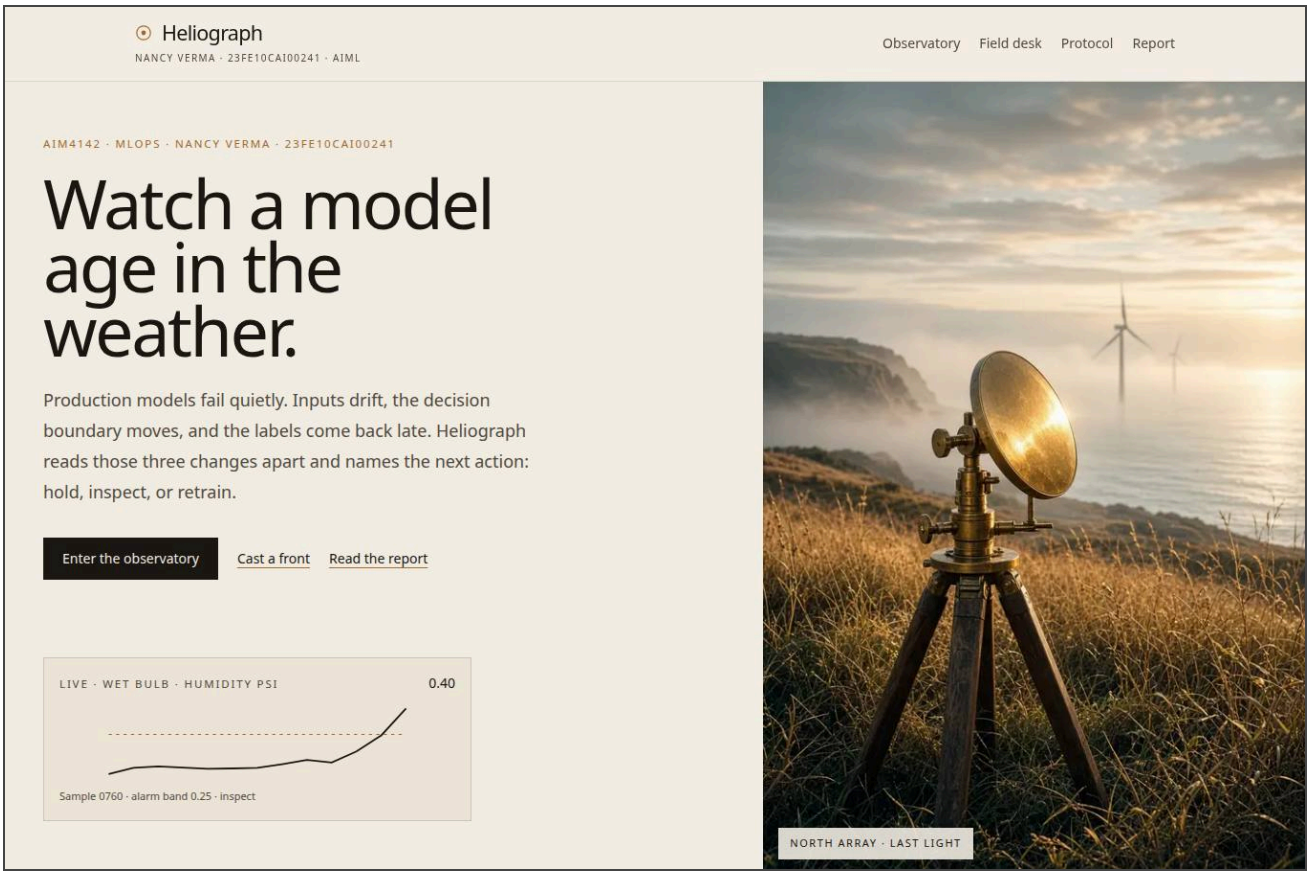


Figure 1. Home page of the deployed project.

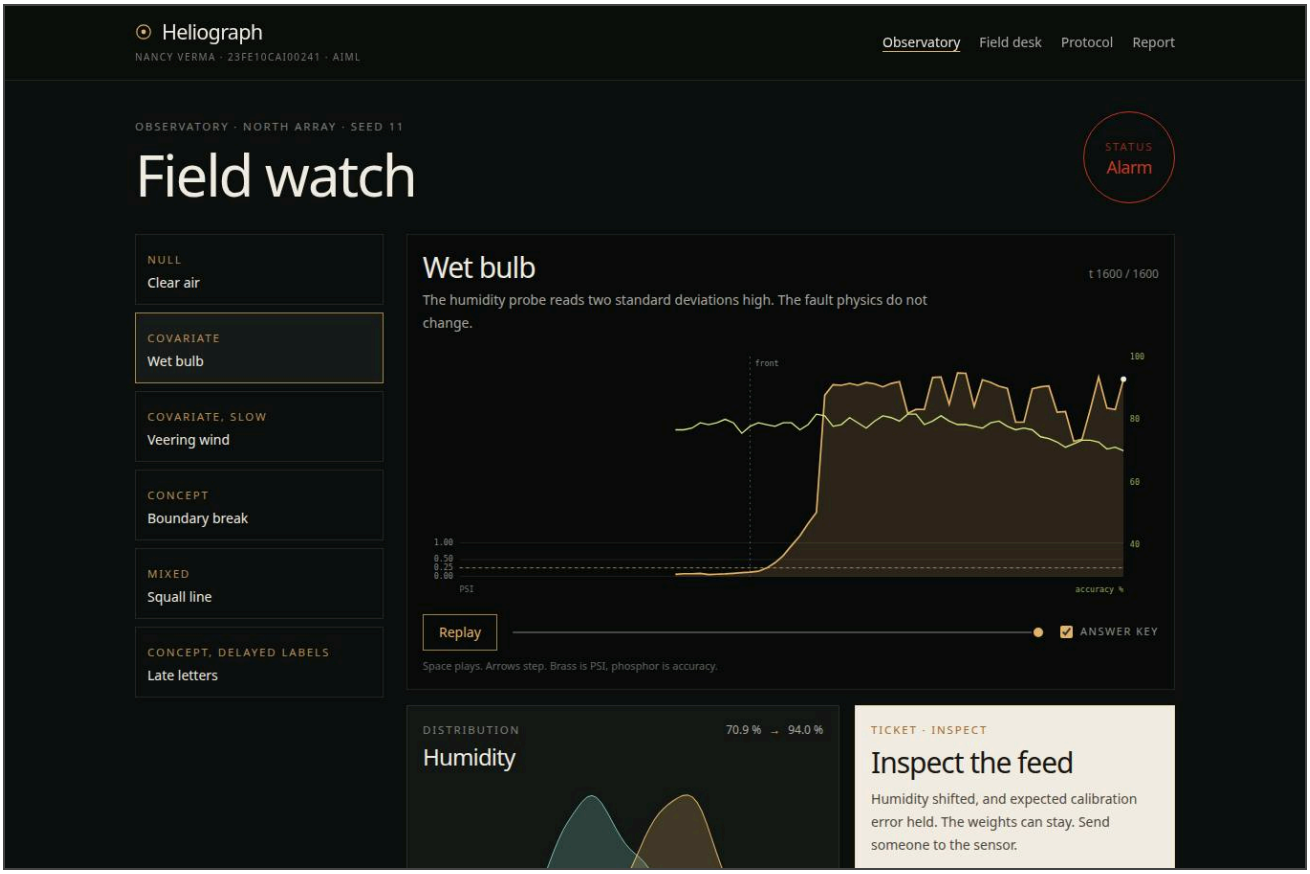


Figure 2. Observatory, wet bulb. Humidity has moved and the action is inspect.

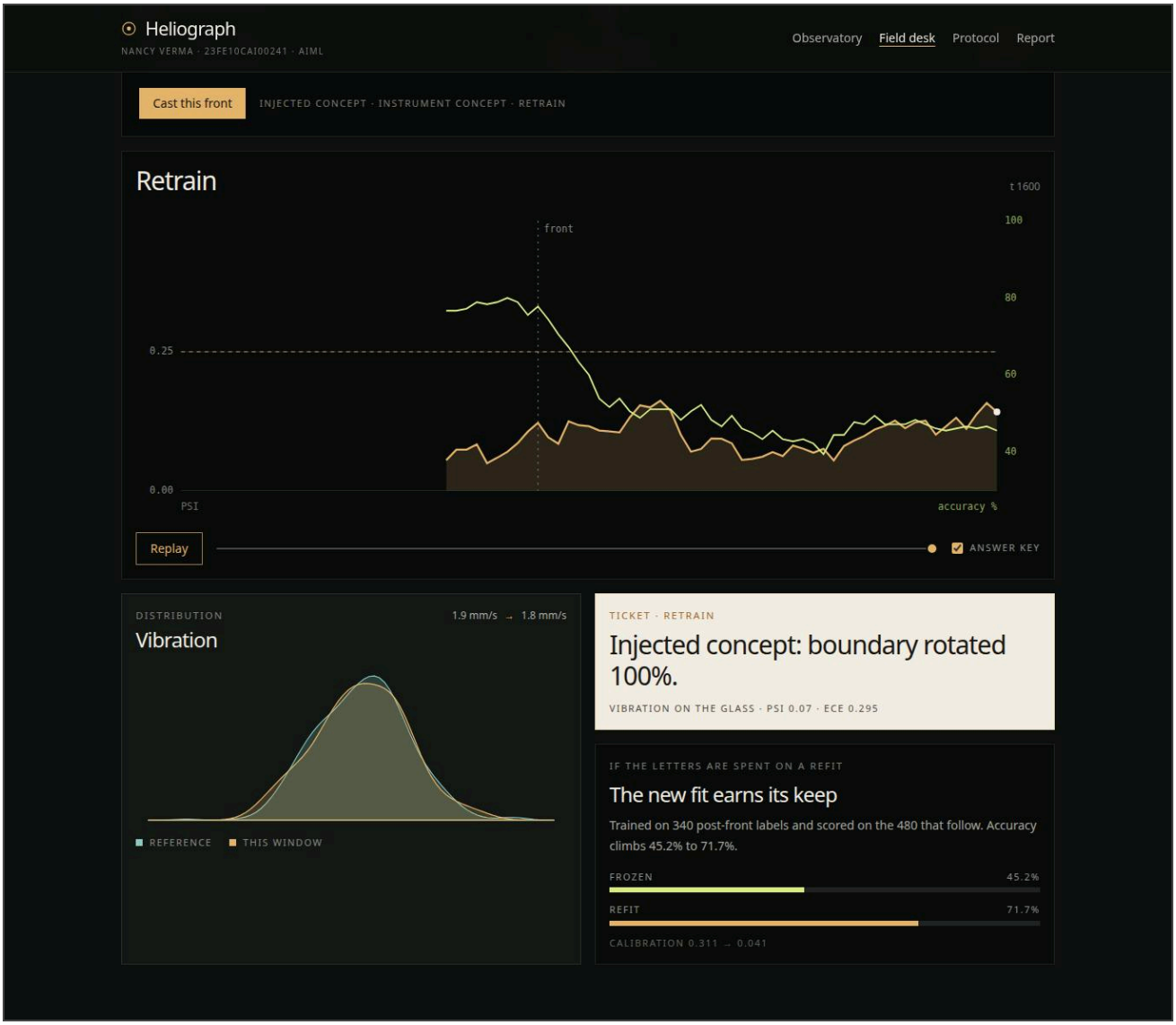


Figure 3. Field desk after a boundary change. A refit on the later labels lifts accuracy from 45.2% to 71.7%.

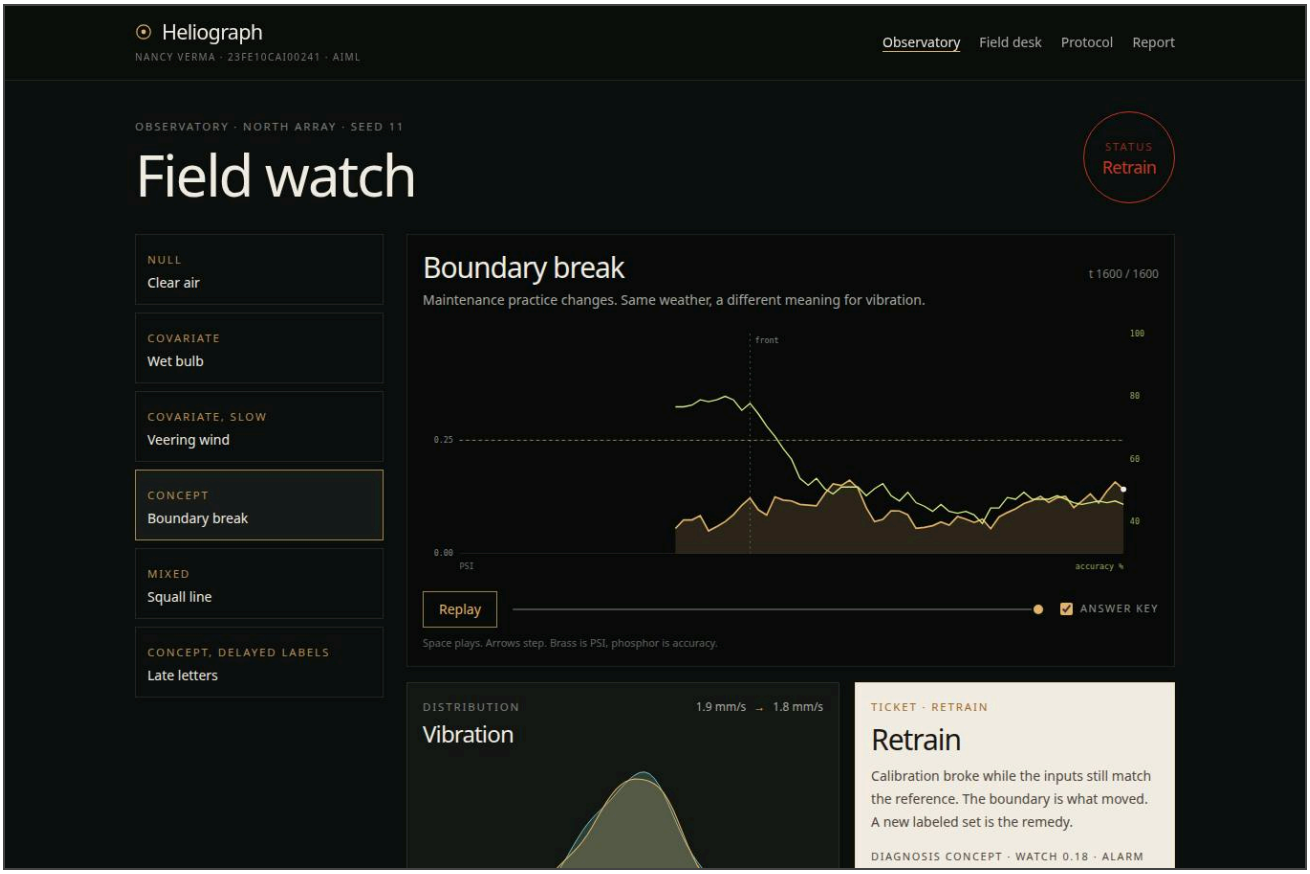


Figure 4. Observatory, boundary break. The vibration densities still overlap, and the action is retrain.

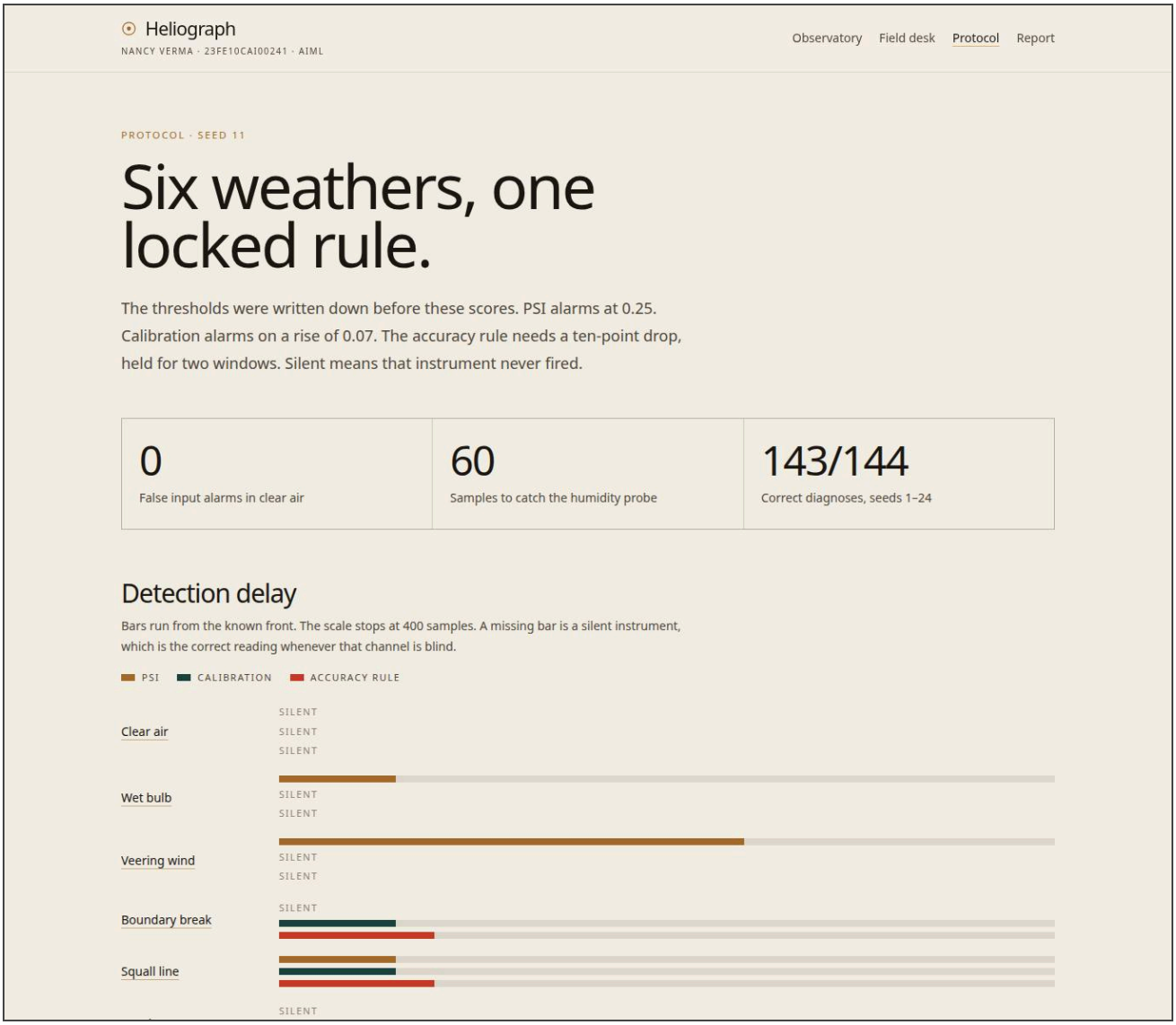


Figure 5. Protocol page for seed 11, with the 143 of 144 result across seeds 1 to 24.

Abstract

This report is for AIM4142 MLOps. I built a small monitoring setup for a gearbox-fault classifier and tested it on simulated wind-turbine data where the type of drift is known. The monitor uses the population stability index on the inputs and expected calibration error on the labels, then picks one action: hold the model, inspect the sensor, or retrain. On seed 11 it gets all six test cases right. Across seeds 1 to 24 it is right on 143 of 144 runs.

1. Problem statement

Once a model is deployed, the data it sees later is not always the data it was trained on. Sometimes only the inputs change. A humidity sensor can start reading higher than it did in training, while the fault logic stays the same. That is covariate shift. Sometimes the relationship changes. The same vibration value may no longer mean the same fault if maintenance practice has changed. That is concept drift. Both can happen together, and the labels that would show it often come in late.

Watching accuracy alone is not enough. Accuracy needs labels, so it is late by definition. It also does not say what to do next. A bad sensor and a changed decision boundary can both reduce accuracy. Retraining on every small input change wastes labels. Waiting until accuracy has already fallen means the model keeps making poor predictions in the meantime.

For this project I used a simulated wind-turbine setting. The classifier predicts a gearbox fault from six sensor readings. Because the data is generated, each scenario has a known ground truth. The monitor is deliberately simple: population stability index (PSI) on the features, and expected calibration error (ECE) on the labels that have arrived. The output is one of three actions. Hold the current weights, inspect the sensor, or retrain.

2. Objectives

- Implement PSI, the two-sample Kolmogorov-Smirnov statistic, Jensen-Shannon divergence, expected calibration error, and a Page-Hinkley test from their definitions.
- Train a logistic regression on a fixed reference window, freeze it, and run six drift scenarios with known causes.
- Compare the combined rule with two simpler baselines: PSI only, and a ten-point drop in accuracy.
- Put the same code on Vercel so the numbers in this report can be checked in the app.

3. Methodology

Each run has 1600 samples and six features: wind speed, vibration, gearbox temperature, rotor speed, humidity, and power. I kept wind, rotor speed, and power correlated, because that is how a turbine actually behaves. Humidity is almost independent of the others, so a fault in that sensor is easy to separate from a real change in the fault logic. Labels are drawn from a logistic model. The true weights are [0.92, 1.08, 0.70, 0.38, 0.02, 0.12] with bias -1.35. When I inject concept drift, those weights change: the vibration coefficient flips sign, humidity starts to matter, and the power coefficient reverses.

The deployed model is a second logistic regression. I trained it with batch gradient descent on samples 0 to 400, with a small L2 penalty, and chose the threshold that maximises F1 on samples 400 to 520. After that the weights are frozen. On the calibration slice the accuracy is 79.2% and the ECE is 0.073. The learned signs match the true signs on the important features. The magnitudes do not match exactly, because wind, rotor speed, and power carry overlapping information and the fit can split the weight between them.

The drift starts at sample 700. Before that, every scenario uses the same stream from seed 11. I check a sliding window of 180 samples every 20 steps. For inputs I take the worst PSI across the six features, using quantile bins from the reference. The usual bands are 0.10 and 0.25. With six features the worst PSI can reach about 0.16 even when nothing has changed, so I set the watch level at 0.18 and kept the alarm at 0.25. I also compute the KS statistic and Jensen-Shannon divergence, but they are only shown. I did not use them in the decision, because a p-value on a large window flags very small shifts.

For concept drift I only use labels that have already arrived. In five scenarios the label is available immediately. In Late letters it arrives 280 samples later. ECE is computed once the window has at least 90 labels. If ECE rises by 0.07 above the reference, that is a concept alarm. I also run a Page-Hinkley test on the squared residual, with delta 0.004 and lambda 18. It is reported next to the decision. The decision itself uses PSI and ECE, which are easier to explain.

I fixed the rule before looking at the scores. If only the inputs alarm, the action is to inspect the sensor and keep the weights. If calibration alarms, the action is to retrain, even if the inputs also moved. Otherwise the action is to hold. The accuracy baseline is separate. It retrains when accuracy falls by 0.10 from the median before the drift and stays down for two windows. That baseline does not know which scenario it is in. The combined rule does not either.

There are six scenarios. Clear air has no drift. Wet bulb is a sudden humidity bias of two standard deviations. Veering wind is a slow rise in wind speed of 1.45 standard deviations over 380 samples. Boundary break changes the logistic weights and leaves the inputs alone. Squall line does both at once. Late letters is the same boundary change, but the labels are delayed by 280 samples.

4. Implementation

I implemented the project as a Next.js app in TypeScript, without a Python runtime or any API key. The statistics and the simulator are in lib/drift and lib/sim. The dashboard, the field desk, the results table, and this report all call those functions, so the numbers here are the same numbers the tests check. Vitest covers the detectors, the six scenarios on seed 11, and a refit on the labels that arrive after the drift. On the field desk I can change humidity bias, wind shift, how far the boundary rotates, and the label delay, and the same rule responds. The app is deployed at https://mlops-project-kohl.vercel.app. The code is at https://github.com/Nancy20052020/Mlops_project.

The main screen replays one scenario. A cursor moves through the stream. One chart shows PSI and accuracy, and another compares the reference distribution with the current window. The recommended action is written out in a short note. A checkbox can show where the drift actually started. The monitor does not use that line. It only sees the features and the labels that have arrived.

True weights and the weights learned on seed 11. Wind, rotor speed, and power overlap, so the fit does not recover the exact values.

Term	Truth	Deployed
Wind speed	0.92	0.65
Vibration	1.08	1.16
Gearbox temp	0.70	0.88
Rotor speed	0.38	0.11
Humidity	0.02	-0.09
Power	0.12	0.56
Bias	-1.35	-1.32

5. Results

On seed 11 the final diagnosis matches the ground truth in all six scenarios. In clear air, PSI, ECE, Page-Hinkley, and the accuracy rule all stay quiet, and the action is hold. In wet bulb, PSI crosses 0.25 at 60 samples after the drift starts, and the feature responsible is humidity. ECE does not rise, so the action is inspect. The sensor moved. The decision boundary did not.

Veering wind is the same kind of change, but slower. PSI crosses 0.25 at 240 samples, which is later than the sudden humidity bias. The final action is again inspect. Accuracy does not fall by ten points. The inputs left the training range, and the probabilities are still reasonable.

Boundary break is the case where an input test cannot help. PSI at the end of the stream is 0.14, which is below the watch level, and the PSI rule never alarms. ECE rises 60 samples after the drift starts. Page-Hinkley follows at 80 samples, and the accuracy rule at 80. Window accuracy falls from about 79.2% to 45.6%. The action is retrain. Squall line trips both PSI and ECE at 60 samples. I label that case mixed, and the action is also retrain.

Late letters repeats the boundary break with a label delay of 280 samples. PSI stays quiet. ECE alarms at 340 samples, Page-Hinkley at 360, and the accuracy rule at 360. All three are after the delay. A method that needs the label cannot detect the change before the label exists. During the wait the inputs still match the reference, which is useful to know, but it does not replace the missing labels.

Samples after the drift starts, seed 11. Silent means that rule never fired. The last column is the action at the end of the run.

System	Truth	PSI	ECE	Page-Hinkley	Accuracy	Call
Clear air	Stable	Silent	Silent	Silent	Silent	Hold
Wet bulb	Covariate	60	Silent	Silent	Silent	Inspect
Veering wind	Covariate	240	Silent	Silent	Silent	Inspect
Boundary break	Concept	Silent	60	80	80	Retrain
Squall line	Mixed	60	60	60	80	Retrain
Late letters	Concept	Silent	340	360	360	Retrain

Comparison with the baselines. PSI catches Wet bulb and Veering wind, and it misses both concept-drift cases. The accuracy rule misses both covariate cases. On Late letters it is as late as ECE, because both of them wait for labels. Using the two signals together is what makes the action specific.

I repeated the six scenarios on seeds 1 to 24. The final diagnosis is correct on 143 of 144 runs. Clear air, Wet bulb, Boundary break, Squall line, and Late letters are correct on every seed. The one miss is Veering wind on seed 23. PSI fires, and much later ECE also rises (delay 660), so the final label is mixed instead of covariate. A long shift in a feature the model actually uses can affect calibration. The rule then asks for a retrain. I left this result in, because changing the threshold to hide one seed would be fitting the test.

6. Conclusion

PSI and calibration are answering different questions. If a sensor leaves the training range and P(Y|X) is unchanged, the practical next step is to check the sensor. Retraining does not fix a biased probe. If the decision boundary changes while the inputs still look normal, PSI will stay quiet and only the labels will show the problem. If those labels are delayed, every method that needs them is delayed by at least that much. The useful part of this project is running the two checks side by side and writing the threshold down before scoring the scenarios.

This is a common MLOps situation. There is a reference window from training, features that can drift, a labelling step that is slow, and a retraining job that costs time. PSI is already used in credit scorecards. Calibration plots are used wherever a probability is acted on. Page-Hinkley is an old test for a change in the mean. I am not claiming a new detector. The point is that using only one of these checks leads to the wrong action.

The data here is a logistic simulation with one change point. Real drift is messier. Categorical features need their own bins, and a neural network can become miscalibrated for reasons this linear model does not show. ECE with five bins is rough in a short window. I raised the PSI watch level from 0.10 to 0.18 because I was taking the maximum over six features. Another feature set would need its own check on data with no drift. Seasonal drift is not in these six scenarios. The same two measurements can still be used on a real pipeline if the reference window comes from training data and the label delay is taken from the actual labelling process.

7. References

1. Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. IEEE International Conference on Big Data, 2017.
2. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. A Survey on Concept Drift Adaptation. ACM Computing Surveys, 46(4), 2014.
3. Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On Calibration of Modern Neural Networks. International Conference on Machine Learning, 2017.
4. Hinkley, D. V. Inference about the Change-Point from Cumulative Sum Tests. Biometrika, 1971.
5. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., and Zhang, G. Learning under Concept Drift: A Review. IEEE Transactions on Knowledge and Data Engineering, 31(12), 2019.
6. Moreno-Torres, J. G., Raeder, T., Alaiz-Rodríguez, R., Chawla, N. V., and Herrera, F. A Unifying View on Dataset Shift in Classification. Pattern Recognition, 45(1), 2012.
7. Page, E. S. Continuous Inspection Schemes. Biometrika, 41(1/2), 100–115, 1954.
8. Rabanser, S., Günnemann, S., and Lipton, Z. Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift. Advances in Neural Information Processing Systems, 2019.
9. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems, 2015.
10. Siddiqi, N. Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring. Wiley, 2006.
11. Webb, G. I., Hyde, R., Cao, H., Nguyen, H. L., and Petitjean, F. Characterizing Concept Drift. Data Mining and Knowledge Discovery, 30(4), 2016.
12. Žliobaitė, I. Learning under Concept Drift: an Overview. arXiv:1010.4784, 2010.